

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
Khoa Toán - Tin học
Bộ môn Ứng dụng Tin học

TOÁN RỜI RẠC 2A

Chương 3. Các thuật toán trên đồ thị

GV: Lê Thị Tuyết Nhung

Bài toán tìm đường đi ngắn nhất

1. Thuật toán Dijkstra
2. Thuật toán Bellman-Ford
3. Thuật toán Floyd

Bài toán đường đi ngắn nhất

- Xét đồ thị $G = (V, E)$. Với mọi cạnh $(u, v) \in E$, ta gọi $\omega(u, v)$ là **trọng lượng** của cạnh (u, v) , $\omega(u, v) = \infty$ nếu $(u, v) \notin E$.
- Độ dài của đường đi từ v_0 đến v_k bằng tổng độ dài các cạnh mà đường đi đi qua.

$$DoDai(v_0, v_1, \dots, v_k) = \sum_{i=1}^k \omega(v_{i-1}, v_i)$$

- Bài toán tổng quát:
 - + Tìm đường đi ngắn nhất từ một đỉnh xuất phát $s \in V$ (đỉnh nguồn) đến đỉnh cuối $t \in V$ (đỉnh đích).
 - + Đường đi như vậy được gọi là đường đi ngắn nhất từ s đến t , độ dài của đường đi $d(s, t)$ được gọi là khoảng cách ngắn nhất từ s đến t .
 - + Nếu như không tồn tại đường đi từ s đến t thì độ dài đường đi $d(s, t) = \infty$.

Mục đích

- Giải bài toán tìm đường đi ngắn nhất từ một đỉnh s tới các đỉnh còn lại của đồ thị
- Áp dụng cho đồ thị với trọng số không âm

Ý tưởng

- Thuật toán được xây dựng dựa trên cơ sở gán nhãn tạm thời cho các đỉnh. Nhãn của mỗi đỉnh cho biết cận trên của độ dài đường đi ngắn nhất tới đỉnh đó.
- Các nhãn này sẽ được biến đổi (tính lại) nhờ một thủ tục lặp. Ở mỗi một bước lặp sẽ có một nhãn tạm thời trở thành nhãn cố định (nhãn đó chính là độ dài đường đi ngắn nhất từ s đến đỉnh đó)

Dijkstra(s){ // $s \in V$ là một đỉnh bất kỳ của $G = (V, E)$

Bước 1: Khởi tạo

$d[s] = 0$; // Gán nhãn của đỉnh s là 0

$T = V \setminus \{s\}$; // T là tập đỉnh có nhãn tạm thời

for ($v \in V$) do // Sử dụng s gán nhãn cho các đỉnh còn lại

$d[v] = a(s, v)$;

$truoc[v] = s$;

endfor

Bước 2: Lặp

while ($T \neq \emptyset$) do

Tìm đỉnh $u \in T$ sao cho $d[u] = \min \{d[z] \mid z \in T\}$;

$T = T \setminus \{u\}$; // Cố định nhãn đỉnh u

for ($v \in T$) do // Sử dụng u , gán nhãn lại cho các đỉnh

if ($d[v] > d[u] + a(u, v)$) then

$d[v] = d[u] + a(u, v)$; // Gán lại nhãn cho đỉnh v

$truoc[v] = u$

endif

endfor

endwhile

Bước 3: Trả lại kết quả

return ($d[s], truoc[s]$);

}

Thuật toán Dijkstra

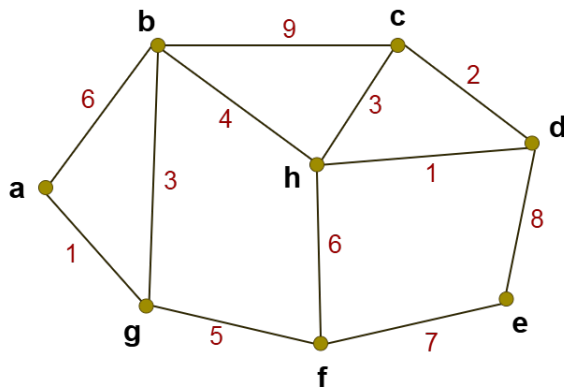
Thuật toán Dijkstra.

Tìm đường đi ngắn nhất từ đỉnh 1 đến những đỉnh khác biết $\omega(e) \geq 0, \quad \forall e \in E$.

- **Bước 1.** Từ đỉnh gốc, khởi tạo khoảng cách tới chính nó là 0, khởi tạo khoảng cách nhỏ nhất ban đầu tới các đỉnh khác là ∞ .
- **Bước 2.** Chọn đỉnh a có khoảng cách nhỏ nhất trong danh sách này và ghi nhận. Các lần sau sẽ không xét tới đỉnh này nữa.
- **Bước 3.** Lần lượt xét các đỉnh kề b của đỉnh a . Nếu khoảng cách từ đỉnh gốc tới đỉnh b nhỏ hơn khoảng cách hiện tại đang được ghi nhận thì cập nhật giá trị và đỉnh kề a vào khoảng cách hiện tại của b .
- **Bước 4.** Quay lại Bước 2 với danh sách khoảng cách đã được cập nhật.
Thuật toán kết thúc khi chọn được khoảng cách nhỏ nhất từ tất cả các điểm.

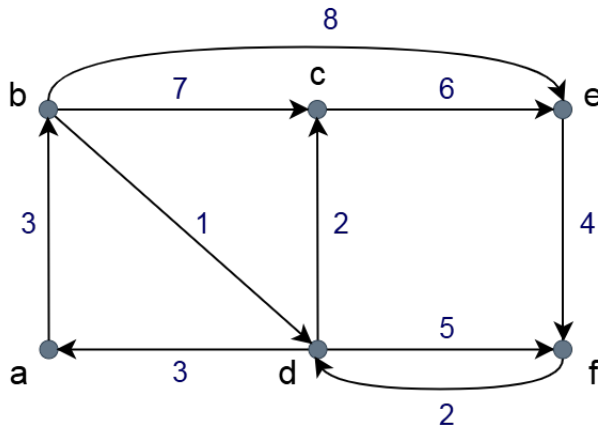
Thuật toán Dijkstra

Ví dụ 1. Tìm đường đi ngắn nhất từ a đến các đỉnh còn lại



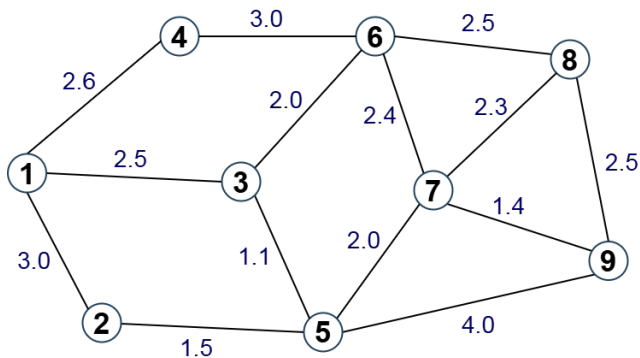
Thuật toán Dijkstra

Ví dụ 2. Tìm đường đi ngắn nhất từ a đến các đỉnh còn lại

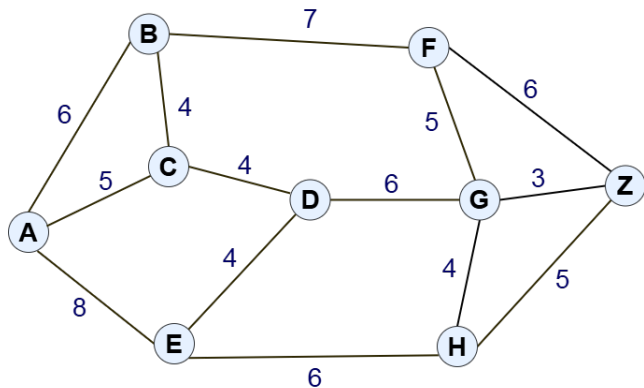


Thuật toán Dijkstra

Bài tập 1. Tìm đường đi ngắn nhất từ 1 đến các đỉnh còn lại



Bài tập 2. Tìm đường đi ngắn nhất từ *A* đến các đỉnh còn lại



Mục đích

- Giải bài toán tìm đường đi ngắn nhất từ một đỉnh s đến các đỉnh còn lại của đồ thị
- Áp dụng cho đồ thị không có chu trình âm (có thể có cạnh âm)

Ý tưởng

- Thuật toán được xây dựng dựa trên cơ sở gán nhãn tạm thời cho các đỉnh. Nhãn của mỗi đỉnh cho biết cận trên của độ dài đường đi ngắn nhất tới đỉnh đó.
- Các nhãn này sẽ được làm tốt dần (tính lại) nhờ một thủ tục lặp. Ở mỗi một bước lặp khi phát hiện $d[v] > d[u] + a(u, v)$, cập nhật $d[v] = d[u] + a(u, v)$.

Bellman-Ford(s) { // $s \in V$ là một đỉnh bất kỳ của đồ thị

Bước 1: Khởi tạo

```
for ( $v \in V$ ) { // Sử dụng  $s$  gán nhãn cho các đỉnh còn lại
     $d[v] = a(s, v)$ ;
     $truoc[v] = s$ ;
}
```

Bước 2: Lặp

```
 $d[s] = 0$ ;
for ( $k = 1; k \leq n - 2; k++$ ) {
    for ( $v \in V \setminus \{s\}$ ) {
        for ( $u \in V$ ) {
            if ( $d[v] > d[u] + a(u, v)$ ) {
                 $d[v] = d[u] + a(u, v)$ ;
                 $truoc[v] = u$ ;
            }
        }
    }
}
```

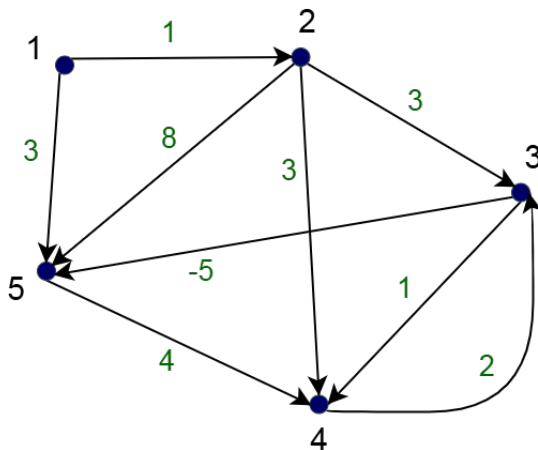
Bước 3: Trả lại kết quả

```
return ( $d[v], truoc[v] : v \in V$ );
```

```
}
```

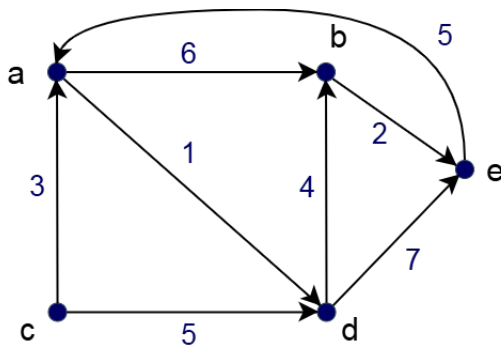
Thuật toán Bellman-Ford

Ví dụ. Áp dụng thuật toán Bellman-Ford tìm đường đi ngắn nhất từ 1 đến các đỉnh còn lại của đồ thị



Thuật toán Bellman-Ford

Bài tập. Áp dụng thuật toán Bellman-Ford tìm đường đi ngắn nhất từ 1 đến các đỉnh còn lại của đồ thị



Mục đích

- Giải bài toán tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh của đồ thị
- Áp dụng cho đồ thị không có chu trình âm (có thể có cạnh âm)

Ý tưởng

- Thực hiện quá trình lặp
- Xét từng đỉnh, với tất cả các đường đi (giữa 2 đỉnh bất kỳ), nếu đường đi hiện tại lớn hơn đường đi qua đỉnh đang xét, ta thay lại thành đường đi qua đỉnh này.

```
Floyd( ){
```

Bước 1: Khởi tạo

```
    for (i = 1; i ≤ n; i++) {  
        for (j = 1; j ≤ n; j++) { // Xét từng cặp đỉnh  
            d[i, j] = a(i, j);  
            p[i, j] = i;  
        }  
    }
```

Bước 2: Lặp

```
    for (k = 1; k ≤ n; k++) {  
        for (i = 1; i ≤ n; i++) {  
            for (j = 1; j ≤ n; j++) {  
                if (d[i, j] > d[i, k] + d[k, j]) {  
                    d[i, j] = d[i, k] + d[k, j];  
                    p[i, j] = p[k, j];  
                }  
            }  
        }  
    }
```

Bước 3: Trả lại kết quả

```
    return (p([i, j], d[i, j] : i, j ∈ V);
```

```
}
```

Thuật toán Floyd

Ví dụ. Áp dụng thuật toán Floyd tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh của đồ thị

